

Improved Priority Exchange Server

Sumon Boiddo

B.Eng Electronic Engineering, Group B, Team 6

Hochschule Hamm-Lippstadt (HSHL), Lippstadt, Germany

Matriculation Number: 2220054

sumon.boiddo@stud.hshl.de

Abstract—This paper presents a comprehensive analysis of the Improved Priority Exchange (IPE) server, an Earliest Deadline First (EDF)-based aperiodic server designed for hard real-time systems. IPE achieves near-optimal aperiodic responsiveness by precomputing the idle-time profile of the optimal Earliest Deadline Late (EDL) server offline and replaying it at runtime through the priority-exchange mechanism of the Dynamic Priority Exchange (DPE) server. This approach avoids EDL's prohibitive runtime recomputation cost while delivering nearly identical aperiodic response times. The paper covers the formal model, the replenishment and exchange algorithm, the schedulability condition, a structured comparison with neighbouring aperiodic servers (PE, DPE, EDL, TBS), a critical evaluation of strengths and limitations, a scalability analysis, and a discussion of suitable and unsuitable embedded systems application scenarios. Proposed extensions address memory reduction, multi-core deployment, and dynamic adaptability. The primary reference is Buttazzo, *Hard Real-Time Computing Systems* [1], Chapter 6.

Index Terms—real-time systems, aperiodic server, EDF scheduling, priority exchange, IPE, embedded systems

I. INTRODUCTION

Hard real-time embedded systems must simultaneously satisfy two conflicting demands. Periodic tasks execute at fixed, predictable intervals and carry hard deadlines whose violation constitutes system failure. Aperiodic tasks arrive unpredictably and must be served as quickly as possible, yet their service must never jeopardise periodic deadlines. Naive solutions perform poorly: background execution makes aperiodic response times unacceptably long under load, while running aperiodic requests at the highest fixed priority risks causing periodic deadline misses [1].

A family of *aperiodic servers* has been developed to resolve this tension. Each server reserves a portion of the processor for aperiodic service while guaranteeing all periodic deadlines. Servers differ in the scheduling algorithm used for periodic tasks, the mechanism by which server capacity is maintained, and the responsiveness achieved for aperiodic requests.

The Improved Priority Exchange (IPE) server [3] occupies a distinctive position in this design space. It combines the dynamic-priority (EDF) scheduling framework of the DPE server with the precomputed idle-time profile of the optimal EDL server, reaching near-optimal responsiveness at a practical

runtime cost. This paper synthesises a technical understanding of IPE (Section II), places it in its scientific context (Section III), and critically evaluates its strengths, limitations, scalability, and applicability (Sections IV–VII). AI usage is documented per the protocol submitted alongside this milestone [6].

II. TECHNICAL BACKGROUND: THE IPE SERVER

A. Problem Setting

A real-time system runs n periodic tasks $\tau_1, \dots, \tau_n$, each characterised by a period T_i , a worst-case execution time C_i , and an implicit deadline $D_i = T_i$. All tasks are activated simultaneously at $t = 0$. The total periodic utilisation is $U_p = \sum_{i=1}^n C_i/T_i$. Under EDF the system is schedulable as long as $U_p \leq 1$, making up to 100% of the processor available, compared with approximately 69% under Rate Monotonic (RM) for large task sets [1]. This larger utilisation bound is the primary reason for choosing EDF as the scheduling algorithm for an aperiodic server.

Alongside the periodic tasks the system receives aperiodic requests at random times with soft (non-hard) deadlines. The challenge is to serve these requests quickly without endangering any periodic deadline.

B. Motivation: EDL, DPE, and the Role of IPE

Two EDF-based aperiodic servers existed before IPE was proposed:

- The *Earliest Deadline Late (EDL)* server [2] is provably optimal—it achieves the minimum possible aperiodic waiting time among all scheduling strategies [5]. However, EDL recomputes the full idle-time schedule at each aperiodic arrival, incurring prohibitive runtime cost that makes it impractical for deployment.
- The *Dynamic Priority Exchange (DPE)* server [3] avoids wasting free processor time by lending its capacity to periodic tasks and saving an equal amount for later reuse. Its replenishment strategy does not, however, exploit the EDL idle-time structure, so it falls noticeably short of optimality.

IPE's core contribution is conceptually simple: the idle-time pattern of the EDL schedule is deterministic and repeats every



Fig. 1. Design lineage of the IPE server. PE introduced the priority-exchange idea under RM; DPE moved it to EDF; IPE added the precomputed EDL idle-time profile [3].

hyperperiod $H = \text{lcm}(T_1, \dots, T_n)$. This pattern can therefore be computed *once, offline*, before the system starts, and stored in compact arrays. At runtime IPE replays these precomputed values through the DPE mechanism, achieving near-optimal responsiveness without EDL's runtime cost [1].

Figure 1 illustrates the design lineage from PE through DPE to IPE.

C. Formal Model

The availability function $\omega(t)$ describes processor idleness:

$$\omega(t) = \begin{cases} 1 & \text{if the CPU is idle at time } t, \\ 0 & \text{if the CPU is busy at time } t. \end{cases} \quad (1)$$

The total idle time in an interval $[t_1, t_2]$ is:

$$\Omega(t_1, t_2) = \int_{t_1}^{t_2} \omega(t) dt. \quad (2)$$

Among all EDF strategies, EDL maximises $\Omega(0, t)$ for every t , meaning it releases processor time as early as possible [5]. Because free time comes earlier, arriving aperiodic requests are served sooner. IPE precomputes this EDL idle-time profile for one hyperperiod and stores it in two arrays:

- $E = (e_0, e_1, \dots, e_p)$: the start times of idle intervals within one hyperperiod;
- $D = (\Delta_0, \Delta_1, \dots, \Delta_p)$: the corresponding durations of those intervals.

The IPE server is then defined by the following rules [3]:

- 1) The server starts with capacity 0.
- 2) At each time instant $t = e_i + kH$ ($k = 0, 1, 2, \dots$), server capacity is replenished by Δ_i units.
- 3) The server always runs at the highest priority in the system.
- 4) All remaining rules—serving aperiodic requests, exchanging and saving capacity, and spare-time reclaiming—are inherited from the DPE server.

D. Schedulability Condition

The periodic tasks together with the IPE server are schedulable—no hard deadline is ever missed—if and only if:

$$U_p \leq 1. \quad (3)$$

The server automatically receives the remaining bandwidth $1 - U_p$ for aperiodic service [3]. This is an exact EDF condition, not a heuristic bound.

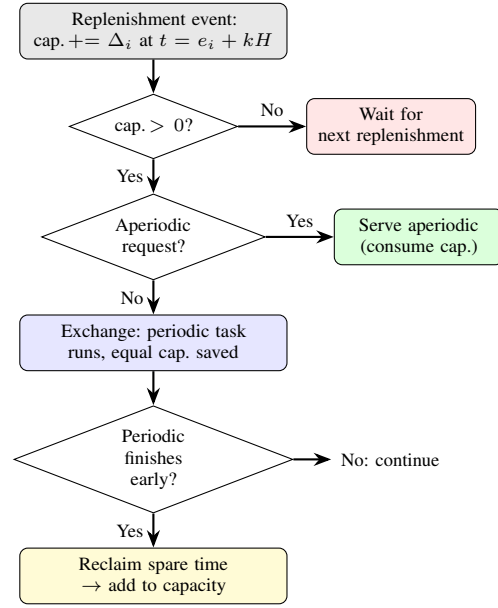


Fig. 2. IPE server operational cycle. At each replenishment the server decides whether to serve an aperiodic request, exchange capacity with a periodic task, or wait. Spare time from early-finishing periodic tasks is reclaimed [1].

E. Operating Algorithm

Figure 2 shows the IPE operational cycle. Four rules govern the server's behaviour at each scheduling event:

- Replenishment:** At each precomputed instant $t = e_i + kH$ the server capacity increases by Δ_i units, exactly mirroring the stored EDL idle time.
- Exchange:** When capacity is available but no aperiodic request is waiting, the capacity is exchanged: a periodic task executes immediately, and an equal amount of capacity is saved for future use. No CPU time is wasted.
- Consumption:** When an aperiodic request arrives and capacity is positive, the request executes immediately at the highest priority, consuming capacity at a one-to-one rate. When capacity reaches zero, aperiodic service is suspended until the next replenishment.
- Spare-time reclaiming:** If a periodic task finishes before its worst-case execution time, the remaining slack is added to the server's capacity. This is safe because the time was already reserved for the periodic task and imposes no additional load [1].

III. SCIENTIFIC CONTEXT: RELATED APPROACHES

A. The Aperiodic Server Landscape

Aperiodic servers belong to two families according to the scheduling algorithm used for periodic tasks [1]. The *fixed-priority* (RM) family includes the Polling Server (PS), the Deferrable Server (DS), the Priority Exchange (PE) server, and the Sporadic Server (SS). The *dynamic-priority* (EDF) family includes the Dynamic Priority Exchange (DPE) server, the Dynamic Sporadic Server (DSS), the Total Bandwidth Server (TBS), the Earliest Deadline Late (EDL) server, and IPE itself.

TABLE I
COMPARISON OF APERIODIC SERVER APPROACHES

Server	Sched.	Resp.	Main limitation
PE	RM	Moderate	Fixed priority; less responsive
DPE	EDF	Good	Periodic replenishment; not optimal
EDL	EDF	Optimal	Heavy runtime recomputation
TBS	EDF	Very good	No idle-time exploitation
IPE	EDF	Near-optimal	High memory (non-harmonic periods)

IPE occupies the end of a clear design lineage: the PE server was extended to EDF as DPE, and DPE was then improved into IPE by grafting in the precomputed EDL idle-time structure [3].

B. Structured Comparison

Table I compares IPE with its four nearest neighbours across the scheduling algorithm used, the aperiodic responsiveness achieved, and the principal limitation.

a) *IPE vs. PE*: PE is IPE’s fixed-priority ancestor. Moving the exchange idea to EDF and adding the precomputed EDL idle times delivers substantially faster aperiodic service [3].

b) *IPE vs. DPE*: Both servers share the EDF scheduler and the priority-exchange mechanism. The improvement in IPE lies entirely in the replenishment policy: DPE uses a periodic refill, whereas IPE replenishes at exactly the EDL idle-time instants stored in the E and D arrays and always runs at the highest priority [3].

c) *IPE vs. EDL*: EDL is provably optimal [2]: no scheduling strategy can reduce aperiodic waiting times further. IPE comes extremely close to this bound but falls marginally short in rare cases. The decisive advantage is that IPE’s heavy computation is performed offline, whereas EDL recomputes at every aperiodic arrival. IPE therefore trades a negligible margin of optimality for a dramatically lower runtime cost, paying the difference in memory [1].

d) *IPE vs. TBS*: TBS assigns each aperiodic request a deadline and schedules it directly under EDF with negligible overhead and no stored tables [1]. It is simpler and lighter than IPE but does not exploit the EDL idle-time structure. Under heavy aperiodic load, IPE’s near-optimal responsiveness is noticeably better than TBS, at the cost of offline precomputation and the memory required for the E and D arrays.

C. Engineering Trade-offs

Four trade-offs characterise IPE’s design position:

Optimality vs. practicality. EDL is theoretically optimal but practically infeasible due to runtime cost. IPE sacrifices a small margin of optimality to become deployable, performing the heavy work once at design time.

Runtime cost vs. memory cost. Precomputing the idle-time schedule shifts computation from runtime to the offline phase.

The benefit is low runtime overhead; the cost is the memory required for the E and D arrays, which grows with the hyperperiod H .

Performance vs. flexibility. Because IPE is tailored to a fixed, fully-known periodic task set, it cannot accommodate a changing task set at runtime. Static high performance is purchased at the expense of dynamic adaptability.

Near-optimal performance vs. simplicity. Compared with TBS, IPE is more responsive but considerably more complex to implement and deploy, requiring both the offline precomputation step and the priority-exchange bookkeeping.

IV. CRITICAL EVALUATION

A. Concrete Strengths

IPE’s foremost strength is *near-optimal aperiodic responsiveness*. By replaying the precomputed EDL idle-time profile, IPE serves aperiodic requests almost as fast as the optimal EDL server and always at the highest system priority when capacity is available [1]. At the same time, its *runtime overhead is low*: the work at each scheduling event consists only of a capacity update and a table lookup, comparable in cost to DPE [3].

The *priority-exchange mechanism* ensures no CPU time is wasted: when no aperiodic request is pending, the server lends its capacity to periodic tasks and saves an equal amount for future use, and spare-time reclaiming further increases effective processor utilisation. Finally, IPE provides a *formal, exact schedulability guarantee* under EDF (equation (3)), not a heuristic bound [3].

B. Concrete Limitations and Risks

IPE’s main weakness is *high memory cost*. The arrays E and D must cover one full hyperperiod $H = \text{lcm}(T_1, \dots, T_n)$. When task periods are non-harmonic, H can grow exponentially with the number of tasks, making the tables impractically large [1]. This is IPE’s principal scalability bottleneck and the most significant obstacle to deployment on memory-constrained embedded devices.

A second major limitation is the *requirement for a static task set*. IPE presupposes that all periodic task parameters are known at design time and remain fixed throughout operation. Any runtime modification to the task set invalidates the precomputed arrays and removes all schedulability guarantees.

Additional limitations include the *need for an EDF-capable operating system* (excluding fixed-priority platforms such as AUTOSAR-based systems), *implementation complexity* relative to simpler servers such as TBS, and the fact that IPE is not *fully optimal*—the EDL server can be marginally better in rare cases [3].

C. Hidden and Potentially Unrealistic Assumptions

Several assumptions underlying the basic IPE model deserve scrutiny in practical deployments:

- *Accurate WCET estimates*. IPE assumes each periodic task’s worst-case execution time is known exactly. In practice WCET is hard to bound tightly, and overestimates lead

to unnecessary capacity reservation while underestimates can cause deadline misses.

- *Full preemptibility.* The model assumes any task can be preempted instantly. Real systems contain non-preemptible critical sections, interrupt handlers, and device-driver code that introduce blocking delays not captured by the basic model.
- *Implicit deadlines and synchronous release.* IPE assumes $D_i = T_i$ and simultaneous task activation at $t = 0$. Systems with constrained or arbitrary deadlines, or staggered phase offsets, require extended analysis.
- *Single processor.* The model is defined exclusively for one CPU. Modern embedded systems are commonly multi-core, and the basic IPE formulation does not extend to them.

V. SCALABILITY ANALYSIS

IPE scales well in *time* but poorly in *memory*. At runtime the server's activity per scheduling event is bounded and independent of the number of tasks: it consists of a table lookup, a capacity update, and optionally a spare-time reclaim. The offline computation of the E and D arrays grows with the hyperperiod but is a one-time design-time cost that does not affect runtime performance.

Memory, by contrast, scales with the hyperperiod. For harmonically-related periods the hyperperiod equals the largest period, so the tables remain small. For non-harmonic periods the hyperperiod is the least common multiple of all task periods, which can grow by orders of magnitude with each additional task, making IPE impractical on memory-constrained devices. Memory is therefore IPE's hard scalability limit.

Overall, IPE is best suited to systems with a small number of harmonically-related tasks and does not scale to large or non-harmonic task sets on limited-memory hardware.

VI. APPLICATION DISCUSSION

A. Suitable Scenario: Robot Motor Controller

A robot motor controller is a strong match for IPE. Its fixed control loops—for example sensor reading at 1 kHz and actuator update at 500 Hz—are harmonically related, known at design time, and carry hard deadlines. Alongside these, the controller receives soft aperiodic events such as operator commands, status queries, and diagnostic messages.

IPE fits this scenario because the periodic task set is static and fully known (enabling offline precomputation of the E and D arrays); the harmonic periods keep the hyperperiod small (keeping the tables compact); the controller runs on a single processor under an EDF scheduler; and IPE guarantees all hard control deadlines while serving aperiodic commands with near-optimal speed.

B. Unsuitable Scenario: AUTOSAR-Based ECU

An automotive engine-control unit (ECU) running an AUTOSAR-based real-time operating system illustrates where IPE fails. AUTOSAR mandates fixed-priority scheduling (RM-style), providing no EDF scheduler and therefore no platform

on which IPE can run. Additionally, AUTOSAR ECUs frequently modify their active task sets at runtime through mode switches and optional function-block activation, which would immediately invalidate IPE's precomputed arrays.

On such a platform, fixed-priority aperiodic servers—the Polling Server, the Deferrable Server, or the Sporadic Server—are the appropriate choice. More generally, IPE is unsuitable for any system with a dynamic task set, a non-EDF scheduler, non-harmonic periods on a memory-limited device, or a multi-core processor.

VII. OPEN QUESTIONS AND PROPOSED EXTENSIONS

The limitations identified above motivate several directions for future work. These are proposed extensions based on the analysis in this paper and are not claims about existing published solutions.

- *Reduced-memory IPE.* Approximate or compress the E and D arrays to make IPE deployable on systems with non-harmonic periods and limited memory, while quantifying the responsiveness penalty introduced by approximation. This directly addresses IPE's principal limitation.
- *Multi-core IPE.* Extend the single-processor IPE model to multi-core architectures, where per-core servers must coordinate capacity exchange without excessive synchronisation overhead.
- *Adaptive IPE.* Allow incremental online updates to the precomputed arrays when the task set changes at runtime, making IPE applicable to semi-dynamic systems without full offline recomputation.
- *Robust IPE under WCET uncertainty.* Introduce a safety margin in the precomputed idle-time profile to tolerate imprecise WCET estimates and variable execution times while maintaining the schedulability guarantee.

An open theoretical question is whether a closed-form worst-case response-time bound for aperiodic requests under IPE can be derived, analogous to the response-time analysis available for fixed-priority servers.

VIII. CONCLUSION

The Improved Priority Exchange server addresses a fundamental challenge in hard real-time systems: how to serve unpredictable aperiodic requests as quickly as possible without endangering hard periodic deadlines. IPE achieves this by combining two previously separate ideas—the precomputed idle-time profile of the optimal EDL server and the priority-exchange bookkeeping of the DPE server—into a mechanism that is near-optimal in responsiveness and practical in runtime cost.

IPE's strengths are its near-optimal aperiodic service, low runtime overhead, CPU-efficient capacity management, and exact formal schedulability guarantee. Its principal weakness is memory consumption, which scales with the hyperperiod and becomes prohibitive for non-harmonic task sets on memory-constrained devices. Additional constraints—a static task set, a single processor, and an EDF-capable scheduler—limit its

applicability to static embedded systems with harmonic periods and sufficient memory.

Future work should focus on reducing the memory footprint, extending the model to multi-core platforms, and enabling online adaptation to changing task sets. These extensions would broaden IPE's applicability while preserving its central advantage of near-optimal aperiodic responsiveness at practical runtime cost.

REFERENCES

- [1] G.C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. Springer, 2011, ch.6 (Dynamic Priority Servers).
- [2] M. Spuri and G. Buttazzo, "Efficient Aperiodic Service under Earliest Deadline Scheduling," in *Proc. IEEE Real-Time Systems Symposium (RTSS)*, 1994.
- [3] M. Spuri and G. Buttazzo, "Scheduling Aperiodic Tasks in Dynamic Priority Systems," *Real-Time Systems*, vol. 10, no. 2, March 1996.
- [4] J. Lehoczky, L. Sha, and J. Strosnider, "Enhanced Aperiodic Responsiveness in Hard Real-Time Environments," in *Proc. IEEE Real-Time Systems Symposium (RTSS)*, 1987.
- [5] H. Chetto and M. Chetto, "Some Results of the Earliest Deadline Scheduling Algorithm," *IEEE Trans. Software Engineering*, vol. 15, no. 10, 1989.
- [6] S. Boiddo, "AI Usage Protocol, Milestone 4 (Final Seminar Paper): Improved Priority Exchange Server," HSHL, 2026. Protocol version 0.2; JSON, BibTeX, and Jupyter Notebook files submitted with this milestone.